Monitoring, Logging & Observability

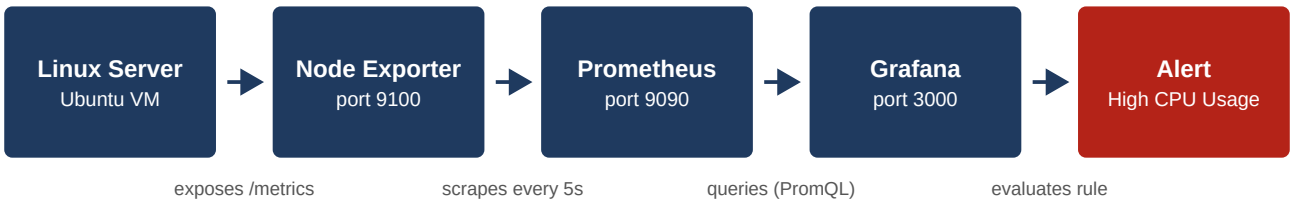
Week 8 Report | DevOps Internship

Oseni Sakariyau Oluwadamilare | 10 October 2026

1. Objective

This report documents the monitoring stack built for Week 8. A Linux server (an Ubuntu virtual machine run with Vagrant and VirtualBox) was monitored with Node Exporter, Prometheus and Grafana. A controlled CPU spike was then generated to practise the full cycle of detecting, investigating and resolving an issue, and an alert was configured to catch it.

2. Monitoring Architecture



Node Exporter exposes host metrics. Prometheus scrapes and stores them as time series. Grafana queries Prometheus to draw dashboards, and Grafana alerting watches the same data and fires when a threshold is crossed.

3. Setup Summary

Component	Role	Configuration
Node Exporter	Exposes CPU, memory, disk and uptime metrics	Service on port 9100; metrics at `/metrics`
Prometheus	Scrapes and stores metrics	Jobs `prometheus` and `node_exporter` (localhost:9090 and localhost:9100); both targets report `up`
Grafana	Visualisation and alerting	Installed from the official apt repository; port 3000 forwarded in the Vagrantfile; Prometheus data source `http://localhost:9090`
Dashboard	Node Exporter Full	Imported from Grafana dashboard ID 1860

Metrics monitored

Metric	PromQL used
CPU usage (%)	`100 - (avg by(instance) (rate(node_cpu_seconds_total{mode="idle"}[1m])) * 100)`
Memory usage (%)	`(1 - node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes) * 100`
Disk usage (%)	`100 - (node_filesystem_avail_bytes{mountpoint="/"} / node_filesystem_size_bytes{mountpoint="/"} * 100)`
System uptime	`time() - node_boot_time_seconds`

4. Linux Logs

Logs explain why a metric changed. The main sources explored were:

Source	What it shows
`/var/log/syslog`	General system messages and service activity
`/var/log/auth.log`	Logins, sudo use and SSH authentication
`/var/log/kern.log` and `dmesg`	Kernel messages, hardware and scheduler warnings
`journalctl -u <service>`	Per-service logs from systemd, such as prometheus and grafana-server

5. ELK Stack Overview

The ELK stack is a log management platform. Where Prometheus and Grafana answer *what is the system doing* with numeric metrics, ELK answers *why* by searching the log text itself.

Tool	Purpose
Elasticsearch	Stores and indexes log data so it can be searched quickly
Logstash	Collects, parses and transforms logs before they are stored (Beats agents such as Filebeat can ship logs too)
Kibana	Web interface to search logs and build visualisations and dashboards

Used together, metrics point to *when* a problem started and logs show *what happened* at that moment. ELK was studied at overview level this week and was not installed on this VM.

6. Alert Configuration

Setting	Value
Rule name	High CPU Usage
Folder	Linux Servers
Query	`100 - (avg by(instance) (rate(node_cpu_seconds_total{mode="idle"}[1m])) * 100)`

Setting	Value
Condition	Is above 80
Evaluation	Group cpu, every 10s, pending period 30s

7. Hands-on Activity: Troubleshooting Flow

Scenario: monitor a Linux server running a web application, create a controlled CPU spike, then investigate and restore normal operation.

Step	Action	Result
1. Baseline	Observed the dashboard during normal operation	CPU mostly idle, memory and disk stable
2. Generate load	`stress --cpu 2 --timeout 900 &`	Two stress workers started
3. Prometheus	Checked targets and the CPU query	node_exporter target up; CPU series rose
4. Grafana	Viewed the CPU panel	Clear spike in CPU busy
5. Logs	Checked journalctl and kernel messages	Kernel RCU stall warnings during heavy load
6. Alert	High CPU Usage rule	Moved from Normal to Pending to Firing
7. Cause	`top` snapshot	Two stress processes in state R at the top; CPU idle 0.0%
8. Restore	`pkill stress`	Load dropped and the alert returned to Normal

8. Troubleshooting Summary

Several real issues came up while building the stack. They are documented here because they are part of the learning.

Issue	Cause	Fix
VM boot timeout and SSH timeout after `vagrant reload`	The VM took about 13 minutes to boot, longer than Vagrant's default wait	Set `config.vm.boot_timeout = 900` in the Vagrantfile
apt sources for Docker, Jenkins and HashiCorp disabled	The Ubuntu release upgrade disabled third-party repositories	Recreated the sources for the new release with `signed-by` keyrings, then upgraded Docker
Minikube cluster stopped and slow to start	The VM was recovering and the API server was slow to report healthy	Started with `--wait-timeout=20m`; the cluster came up and was verified with kubectl
Prometheus job name did not match the task	Job was named `node` instead of `node_exporter`	Renamed the job in prometheus.yml and restarted Prometheus

Issue	Cause	Fix
Stress test results unreliable: a 60 second test took 252 seconds	Host CPU starvation of the VM, with kernel RCU stall warnings and high load average while the CPU showed idle	Used a long-running load and captured evidence while it ran; reduced the VM to 1 CPU in the Vagrantfile

Key lesson: metrics only show what the guest sees. When the host starves the VM, timings can mislead, so cross-check metrics with logs.

9. Evidence Screenshots

Figure numbers correspond to the file names in the Evidence folder submitted with this report.

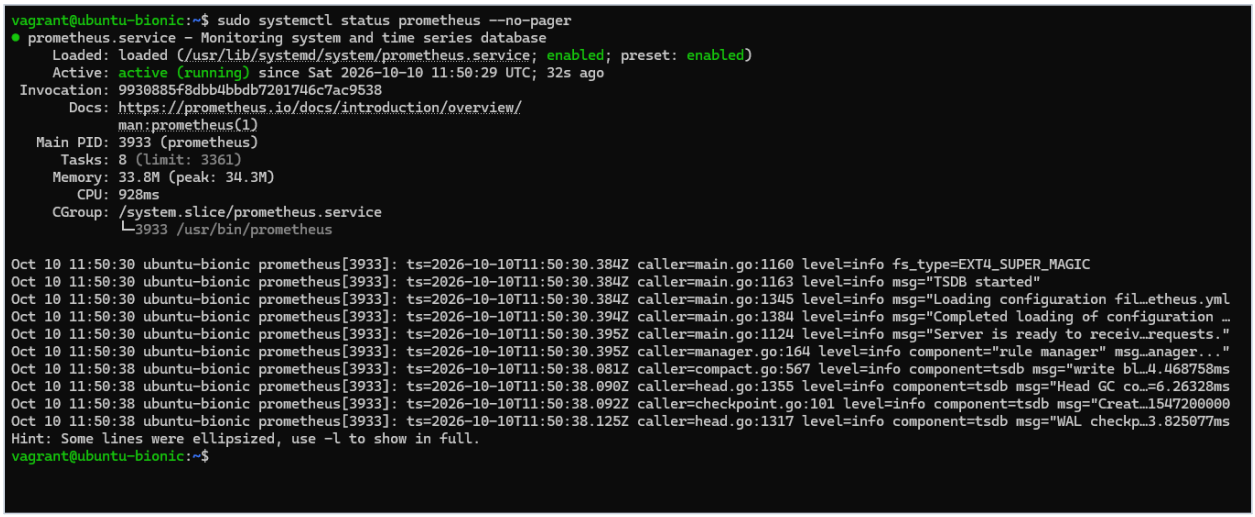


Figure 1. Prometheus running as a systemd service on the monitored host.

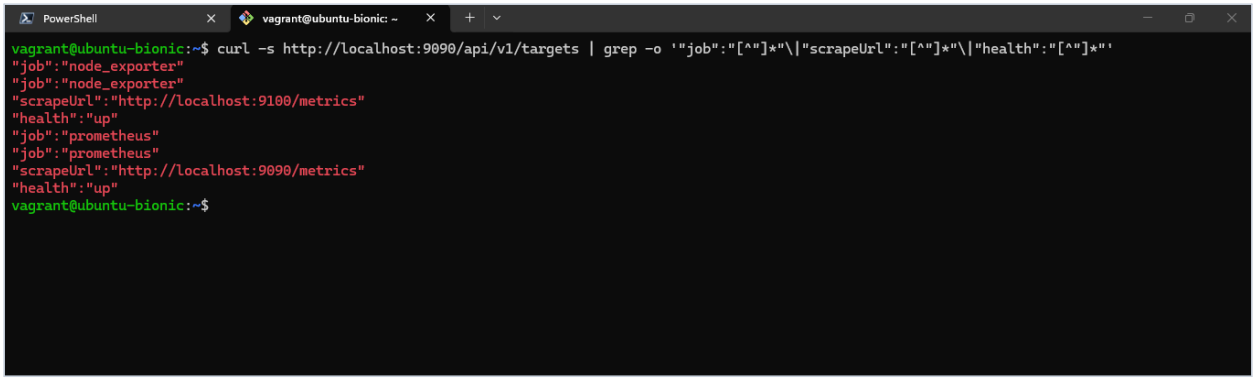


Figure 2. Prometheus targets API: the prometheus and node_exporter jobs both report health up.

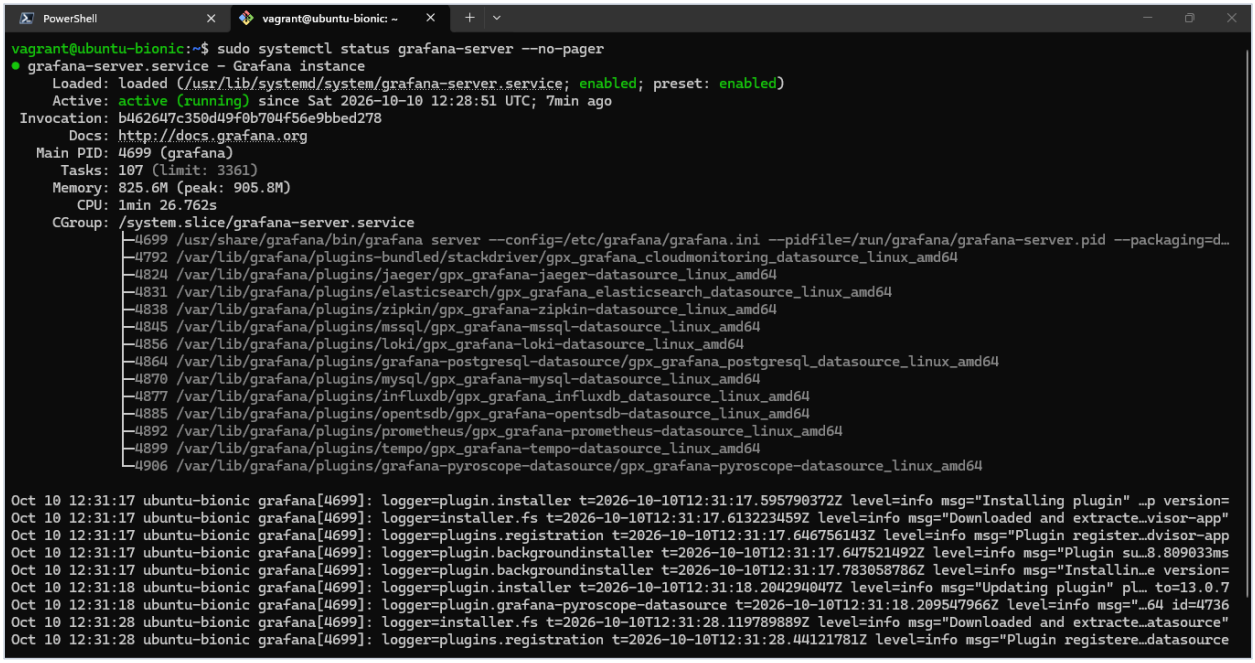


Figure 3. Grafana server active, with the bundled data source plugins loaded.

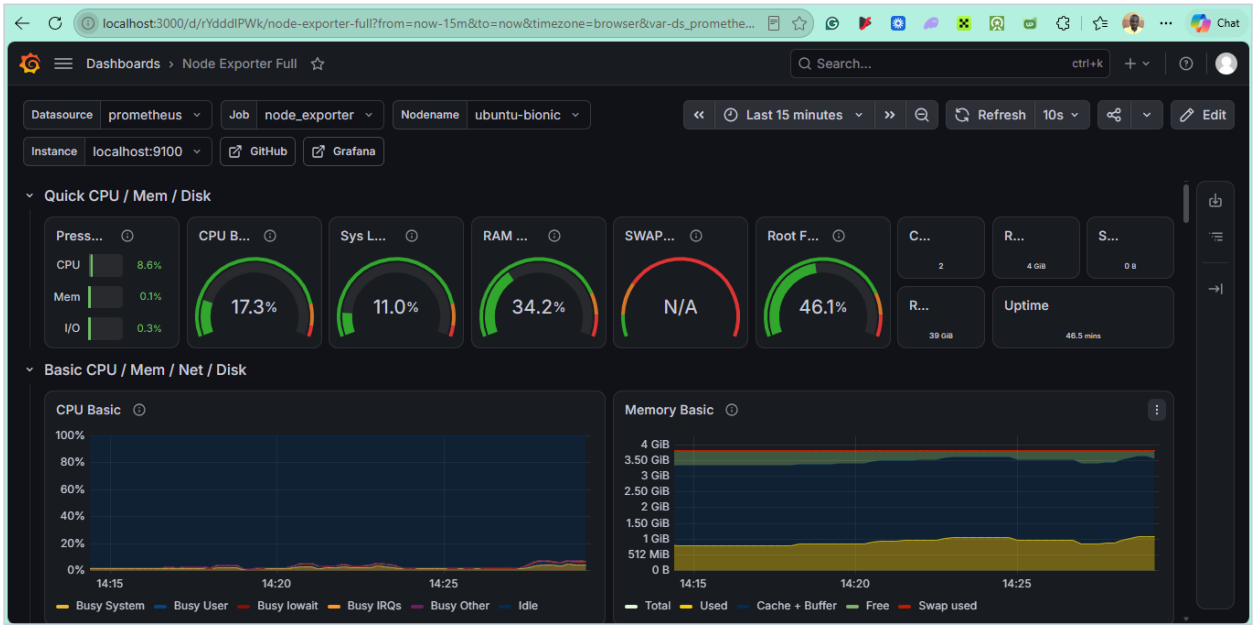


Figure 4. Node Exporter Full dashboard during normal operation: CPU busy 17.3%, RAM 34.2%, root filesystem 46.1%.

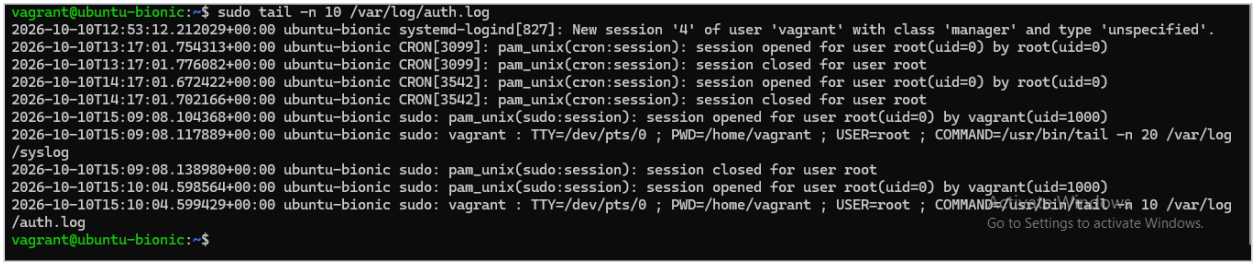


Figure 5. /var/log/auth.log: sudo sessions and authentication events.

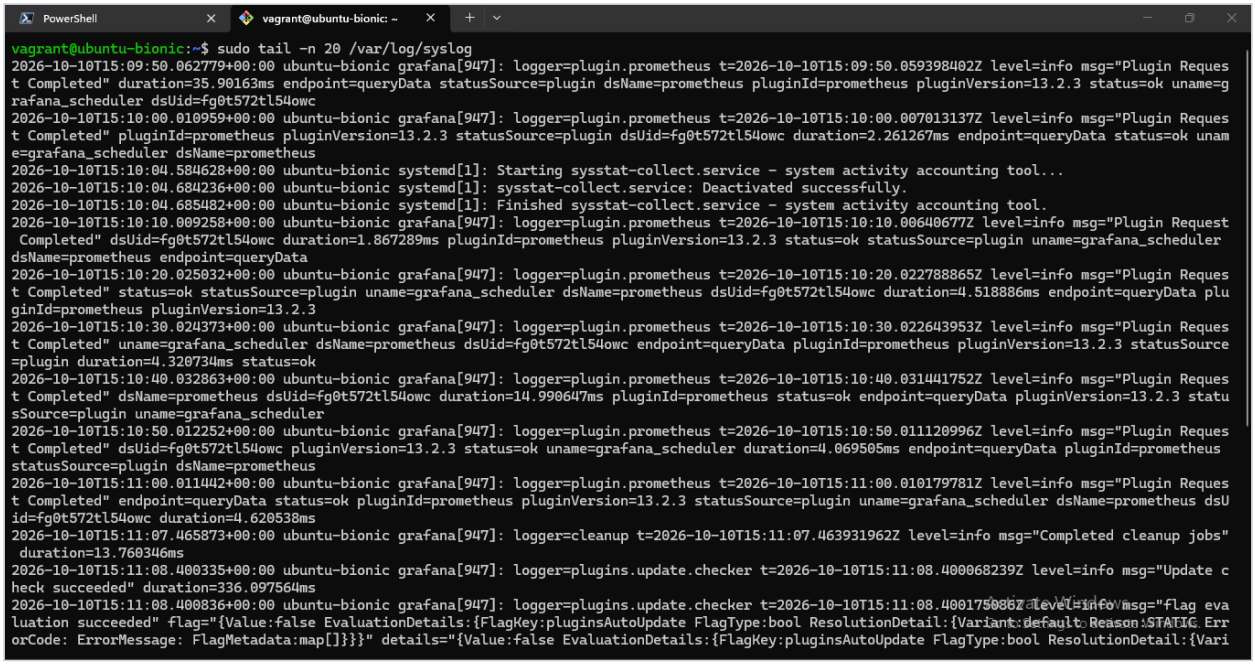


Figure 6. /var/log/syslog: Grafana scheduler queries to Prometheus, and systemd service activity.

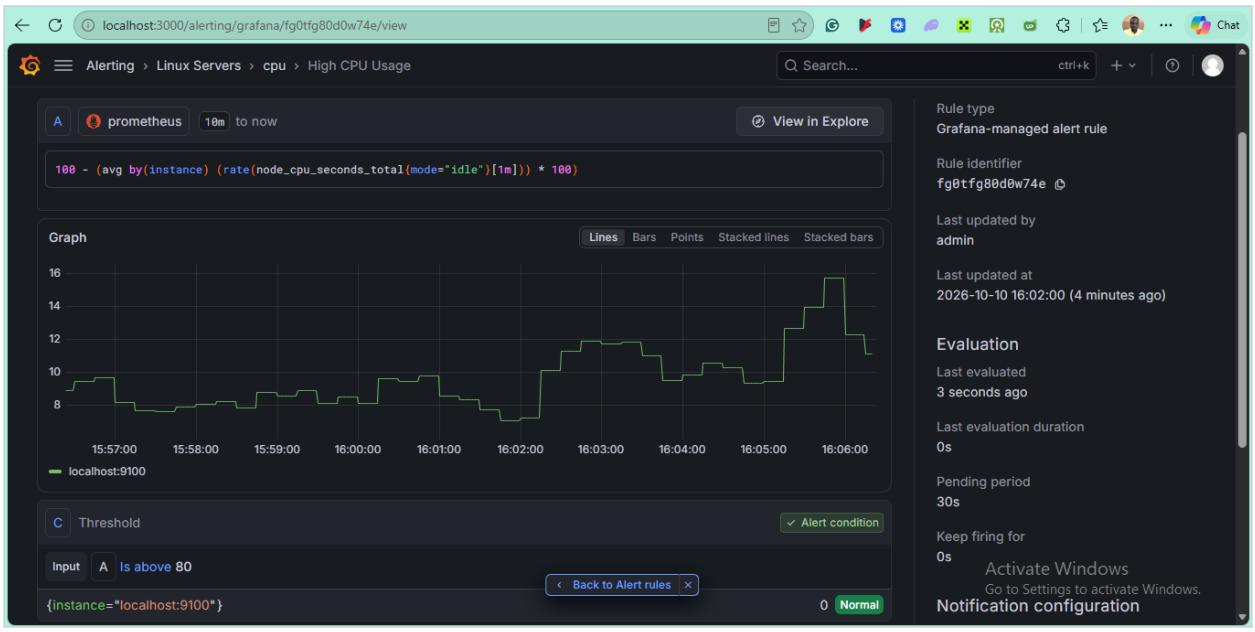


Figure 7. High CPU Usage alert rule: PromQL query, threshold above 80, pending period 30s, state Normal before the test.

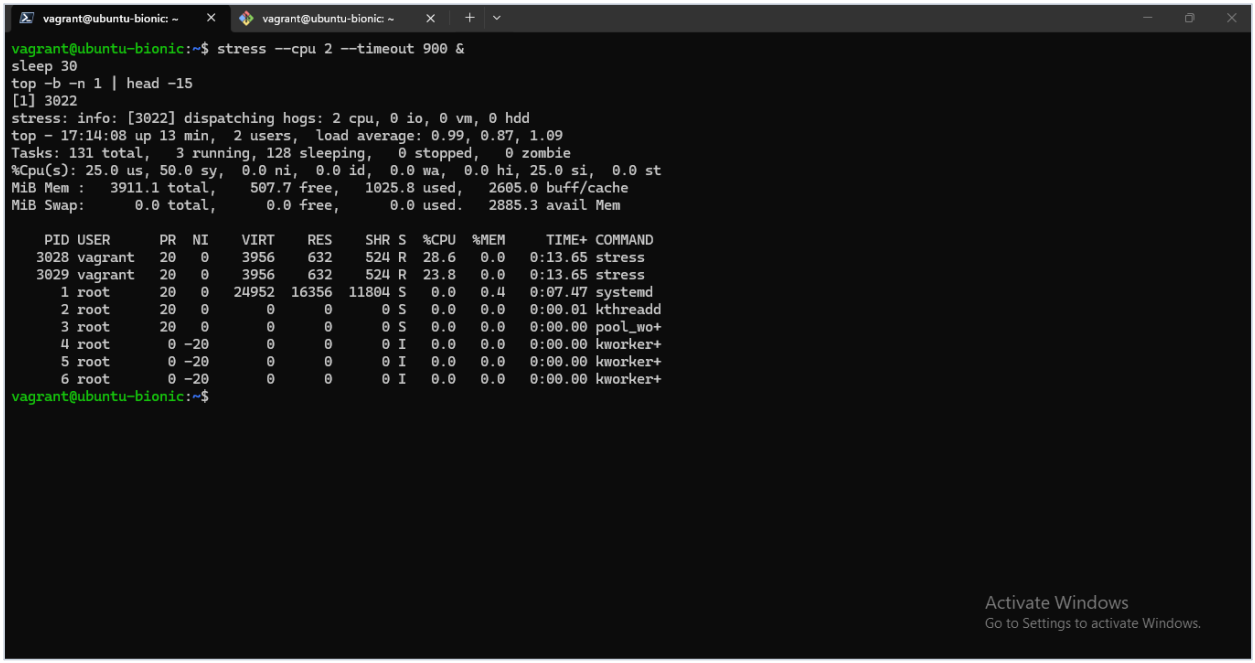


Figure 8. Controlled load generated with stress: two worker processes in state R and CPU idle at 0.0%.

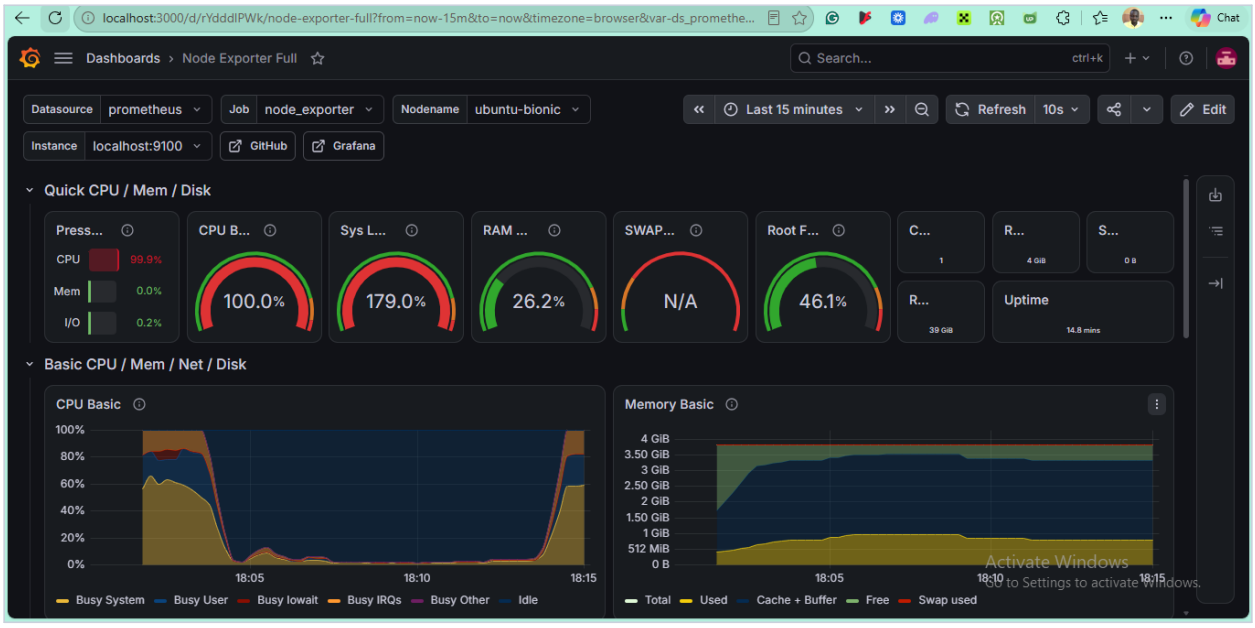


Figure 9. Grafana during the load test: CPU busy at 100% and system load at 179%.

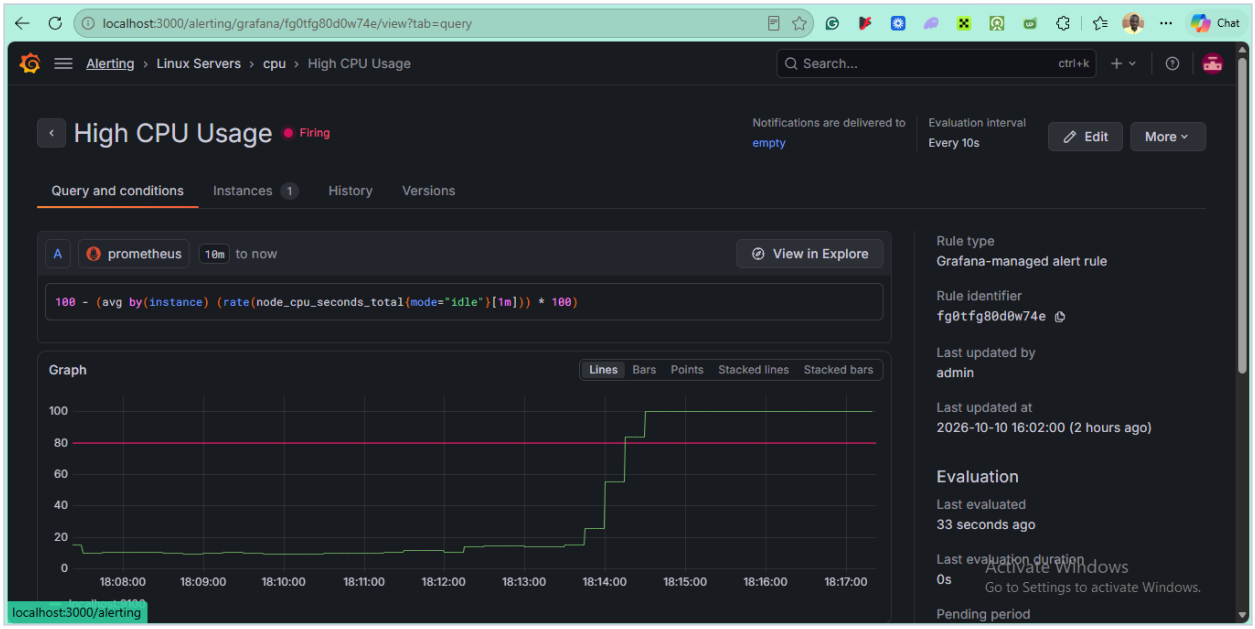


Figure 10. The alert query crossing the 80% threshold line, taking the rule into Firing.

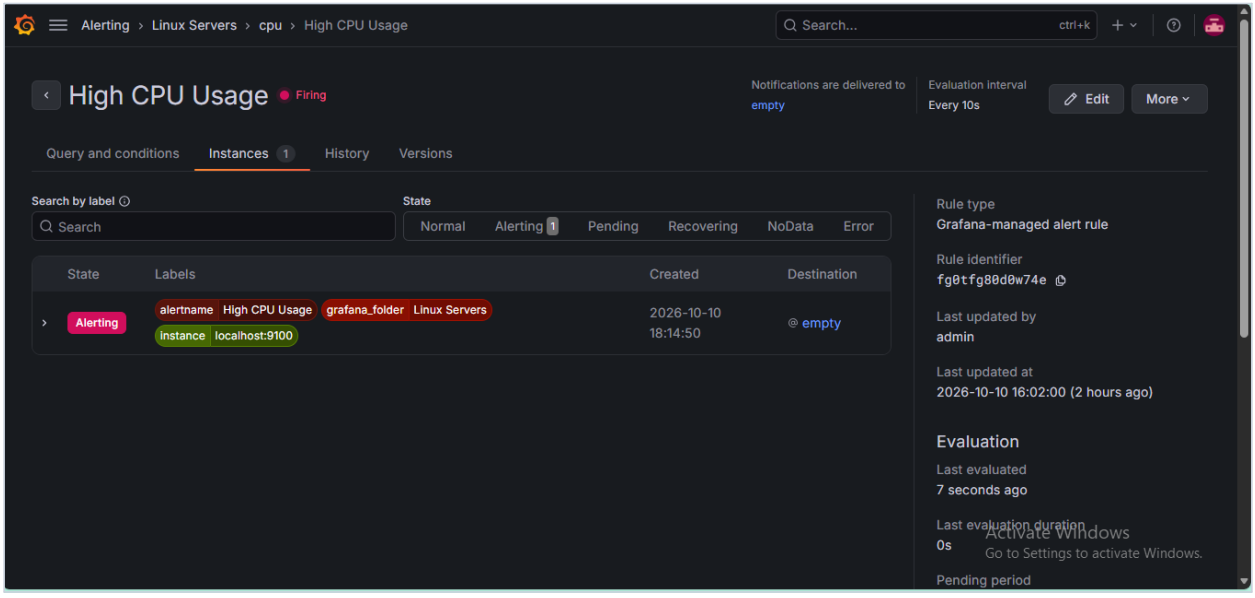


Figure 11. Alert instance in Alerting state, labelled with the rule name, folder and instance.

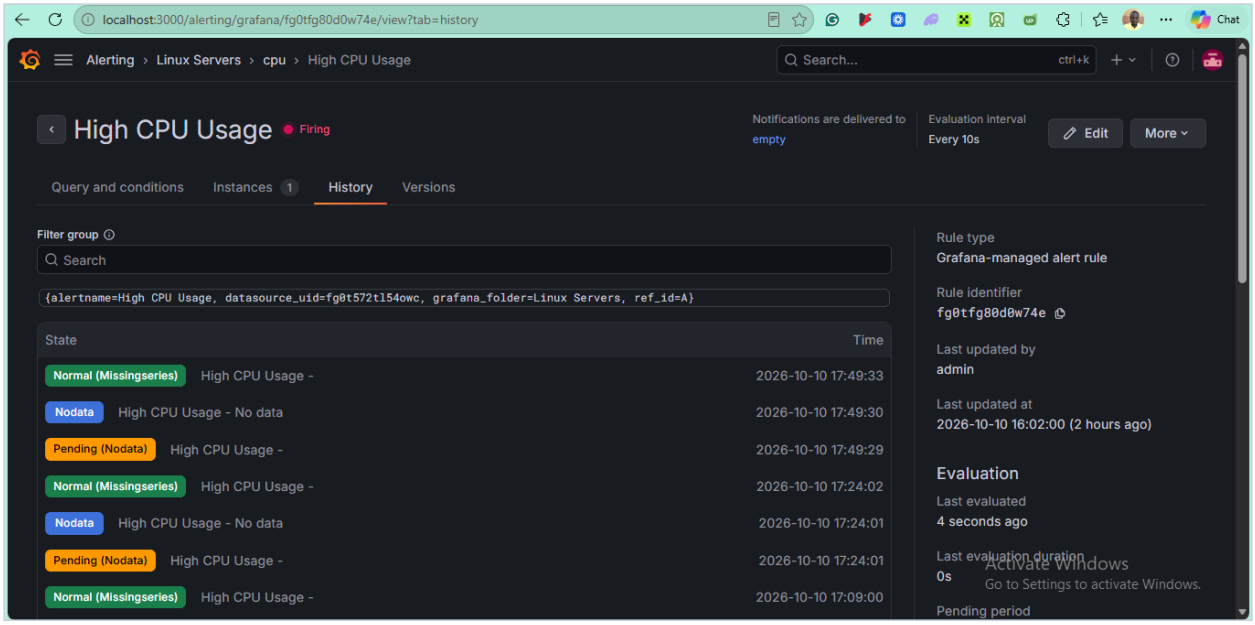


Figure 12. Alert state history showing the transitions recorded by Grafana.

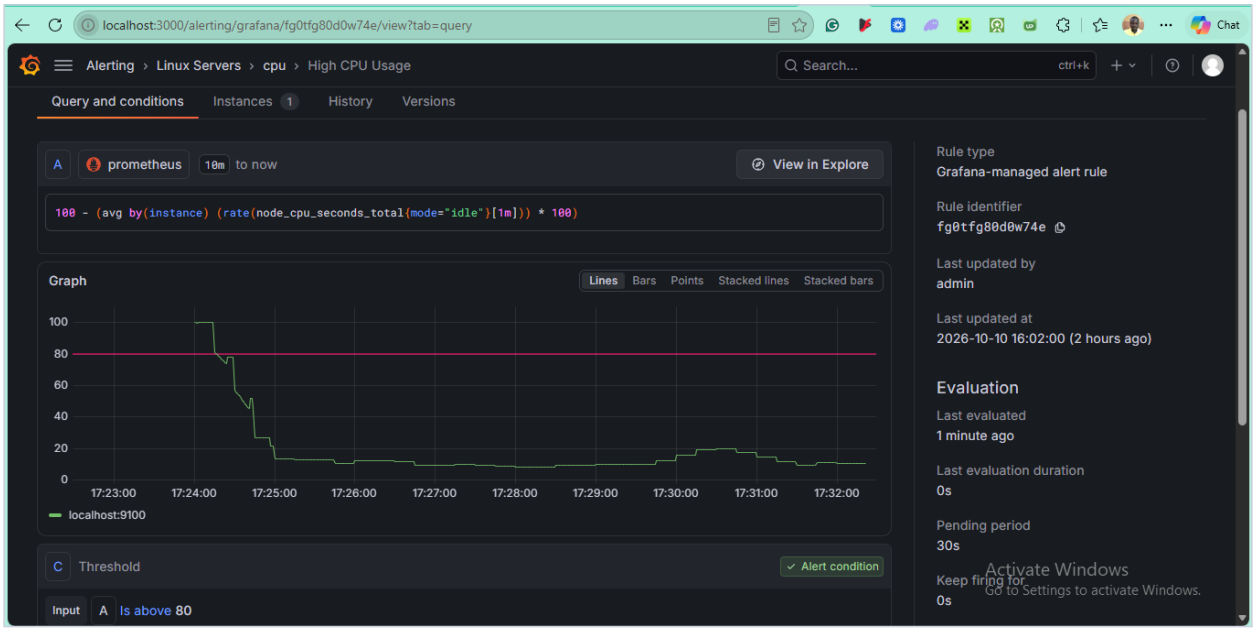


Figure 13. CPU returning to baseline after the load was stopped with pkill stress, and the alert clearing.

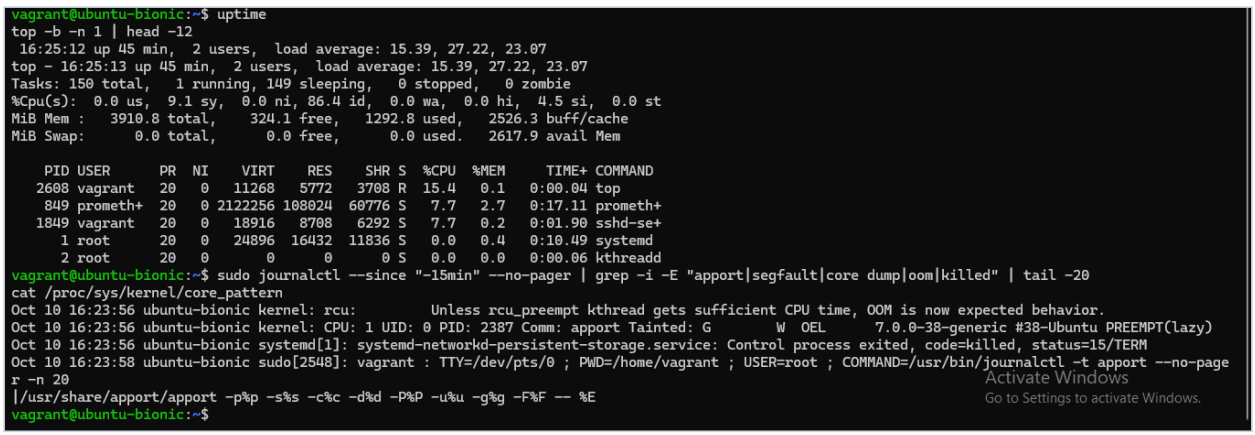


Figure 14. Troubleshooting evidence: load average of 15.39 while the CPU was 86.4% idle, plus the kernel RCU stall warning that identified host CPU starvation of the VM.